Evolutionary Adversarial Reprogramming of Growing Neural Cellular Automata

Aaron Bussche David Magaram Jason Kasdiran
Group 28

June 11, 2026

Abstract

Final Report: Aim for a max 10 pages document, excluding references and contributions of team members. We will not subtract points for small deviations from the 10 pages, but keep in mind that if your report is much longer, there is likely an issue with conciseness / data presentation somewhere!

Introduction

Neural Cellular Automata (NCAs) are small, shared-weight networks that grow images from a single seed pixel and can regenerate after damage, making them a promising model for robust, self-organizing systems. Their perceived robustness has led to NCAs being deployed as a defensive component in larger architectures: Xu et al. [2] use NCA layers as plug-and-play adaptors inside Vision Transformers to make them more resistant against adversarial samples. Whether NCAs themselves can be subverted is therefore not just of theoretical interest but relevant to the security of systems that rely on them.

Randazzo et al. [3] and Cavuoti et al. [4] demonstrated targeted attacks that, for example, can make a generated lizard image lose its tail or change color. However, every published attack on NCAs relies on white-box, gradient-based methods that require full access to the model's weights and backpropagation through the rollout. Robustness claims are so far tested only against this strong threat model. Additionally, focusing on the nature-inspired roots of NCAs, a white-box is not what a realistic adversary, for instance in a biological or bioengineering setting, would plausibly have access to.

We therefore ask: *can a query-only, black-box attacker using evolutionary search reproduce the same targeted reprogramming of a Growing NCA, and how close does it get to the gradient-based ceiling?* We attack the NCA via a 16×16 state-perturbation matrix applied to every cell's 16-dimensional hidden state at every timestep, the same attack surface used by Randazzo et al. Our attacker observes only the final RGB output and optimizes the L2 distance to a chosen target image using CMA-ES, without ever accessing model weights, hidden channels, or gradient information.

If Growing NCAs can be influenced by this attack in a similar way as through white-box approaches, they may be less robust than priorly assumed. Otherwise, they may be safe from more realistic attack types and to be used in public-facing, but not open-weights, ML systems.

Related Work

Mordvintsev et al. [1] introduced the Growing NCA model and showed that, through a sample pool training strategy and exposure to damage during training, the learned local update rules become robust enough to regenerate damaged parts. Due to this robustness, NCAs have also been used defensively: Xu et al. [2] inserted NCA layers between vision transformer blocks to serve as an armor against adversarial attacks.

On the offensive side, Randazzo et al. [3] investigated white-box vulnerabilities of Growing NCAs and showed that, while they are more resistant to local injections, they can still be reprogrammed using global state perturbations applied to every cell at every step. Similarly, Cavuoti et al. [4] demonstrated that injecting a small number of adversarial cells into the grid can drastically alter the grown image. Both studies rely on gradient-based optimization with full access to model weights.

Covariance Matrix Adaptation Evolution Strategy (CMA-ES), introduced by Hansen [5], is an algorithm for non-linear, non-convex black-box optimization that maintains a population of candidates and iteratively shifts them toward high-

fitness regions. While evolutionary black-box attacks are commonly used to evaluate traditional image classifiers, they have not yet been applied as an adversarial tool against cellular automata.

Methodology and Approach

Motivation

To assess the true vulnerability of NCAs under a realistic threat model, we constrain the attacker to a strict black-box setting. The attacker cannot access the internal weights of the pretrained NCA, nor can they perform automatic differentiation through the grid's temporal evolution. Instead, the attacker can only provide an initial state (or intervening perturbation) and observe the final converged visual output. Because the fitness landscape of an NCA rolled out over hundreds of steps is highly non-convex and non-differentiable from a black-box perspective, we utilize CMA-ES. This evolutionary strategy is well-suited for our setup because it is derivative-free, robust to noisy objective functions, and scales well to our specific parameter space.

Experimental Setup

We target a standard pretrained Growing NCA model optimized to generate a specific 2D image from a single seed pixel (e.g., the "lizard" emoji model). The NCA operates on a grid of cells, each possessing a continuous state vector $x \in \mathbb{R}^{16}$.

Following the framework established by Randazzo et al. [3], we restrict our attack to a global, linear state perturbation. At each timestep of the simulation, before the NCA's convolutional update rules are applied, every cell's state vector is multiplied by a global transformation matrix $M \in \mathbb{R}^{16\times16}$. To maintain stability in the cellular dynamics, we restrict M to be a symmetric matrix.

A symmetric 16×16 matrix contains exactly 136 unique parameters (the upper triangular elements, including the diagonal). This 136-dimensional vector space forms the search space for our CMA-ES algorithm. To evaluate the impact of the starting distribution on the evolutionary search, we initialize the population center from three distinct points:

- **The Identity Matrix:** This represents a "zero-damage" starting point, allowing the evolutionary search to incrementally discover minimal perturbations.
- **A Random Matrix:** Initialized via a standard normal distribution to test the algorithm's ability to converge from an arbitrary, highly destructive state.
- **The Baseline Gradient Matrix:** The final output matrix derived from the white-box gradient descent method used by Randazzo et al., establishing a benchmark for convergence speed and local minimum trapping.

For the initial standard deviation (σ_0), which dictates the initial search volume, we conduct a sweep across multiple magnitudes (e.g., $10^{-2}, 10^{-3}, 10^{-4}$). A smaller σ_0 is critical when starting from the identity matrix to prevent immediate catastrophic failure of the NCA rules.

Population size.

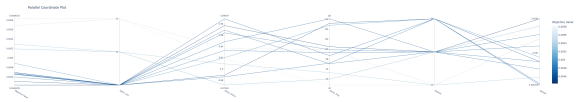

Figure 1: The first optuna sweep, 30 times 20 minutes with early pruning after 5 minutes. This run clearly favored a lot batch size, a population size of 32 or 64 and, multiplied together, a moderate decay (factor/frequency)

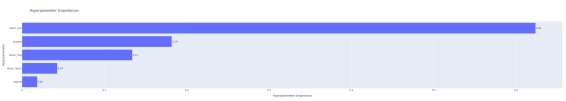

Figure 2

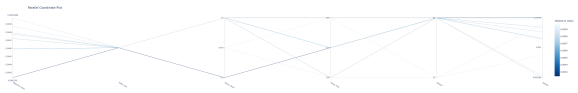

Figure 3: Second optuna sweep with more limited options, winner from the first round re-introduced, 20 times 20 minutes with early pruning after 5 minutes. This run favored popsize 64 over 32, a higher sigma0 and a higher decay frequency (or no decay at all, though a run with decay won overall)

Figure 4

Our final algorithm uses the following parameters: [Insert final chosen parameters here af-

ter running your grid search/ablations, e.g., $\lambda = 64, \sigma_0 = 0.0002$].

sigma0	0.0048
batch_size	4
popsize	64
decay_factor	0.9
decay_freq	175

Evaluation Metrics

While the Mean Squared Error (MSE, or L_2 distance) is utilized as the primary fitness function during the CMA-ES optimization process due to its computational efficiency, it is widely recognized that pixel-wise distances correlate poorly with human visual perception. A perturbation matrix may achieve a low MSE while simultaneously producing an image with unnatural artifacts or structurally disjointed features. Therefore, to rigorously evaluate and compare the outputs of the black-box evolutionary attack against the baseline gradient-based attack, we employ a suite of quantitative metrics focusing on structural, perceptual, and frequency-domain similarities.

Structural Similarity Index Measure (SSIM): Introduced by Wang et al. [7], SSIM evaluates the visual impact of luminance, contrast, and spatial structural dependencies between two images. Unlike MSE, which estimates absolute errors, SSIM considers image degradation as perceived change in structural information. We calculate the SSIM over the RGB channels with a maximum pixel value of 1.0. A higher SSIM value (closer to 1.0) indicates better structural fidelity to the target image.

Learned Perceptual Image Patch Similarity (LPIPS): To quantify semantic and perceptual similarity, we utilize the LPIPS metric [8]. By passing both the target image and the generated NCA output through a pretrained deep convolutional network (in our case, AlexNet) and comparing the internal feature activations, LPIPS effectively measures whether the generated image "looks" semantically identical to the target. Images are normalized to the $[-1, 1]$ range prior to extraction. A lower LPIPS score indicates that the adversarial image is perceptually closer to the intended target.

Frequency-Domain Analysis (2D-FFT): Adversarial reprogramming of cellular automata relies on altering local convolutional updates, which frequently introduces high-frequency structural noise (e.g., checkerboard artifacts) not heavily penalized by spatial loss functions. To quantify these artifacts, we perform a 2D Fast Fourier Transform (2D-FFT) on the grayscale representations of the generated and target images. We compute the mean absolute difference of their log-magnitude spectra:

$$D_F = \frac{1}{H \times W} \sum_{u,v} \left| \log(1 + |\mathcal{F}(\text{img}_{\text{target}})_{u,v}|) - \log(1 + |\mathcal{F}(\text{img}_{\text{grown}})_{u,} \right.$$

where $\mathcal{F}$ denotes the discrete 2D-FFT.

Furthermore, because our targets involve localized morphological changes (such as the removal of a lizard's tail or head), a global Fourier transform may average out highly localized frequency discrepancies. To address this, we implement a partitioned spectral difference metric: the images are divided into a 2×2 grid (four quadrants), and the spectral difference is calculated independently for each block and then averaged. This allows us to precisely capture localized artifacting around the explicitly altered regions of the target shapes.

Losses

Loss function is a mathematical way of quantifying the error between a model's prediction and the actual target value. Similarly, a fitness function evaluates how good a candidate solution is. Both work similarly, just in different directions, and while we usually try to maximize one and minimize the other, in the settings of CMA-ES loss is fitness.

At each generation, every candidate perturbation matrix is evaluated by rolling out the NCA for a batch of states and computing a scalar distance between the generated output and the target image. CMA-ES then uses this to update its search distribution, focusing on lower loss regions on the parameter search space. Therefore, loss function plays an important role in determining what is better during optimization, with different loss functions leading to different results.

5 different loss functions were evaluated in total. All were evaluated on the same color change task across 500 generations using identical parameters found in Table X (batch size = 4, sigma0 = 0.0048, decay factor = 0.9, decay frequency = 175). Losses were selected based on their relevance to this task, with MSE serving as the baseline, and evaluated using the evaluation metrics mentioned in an earlier section.

Mean Squared Error (MSE): MSE measures the average of the squared differences between the actual and predicted values. It is a pixel-wise loss that is applied to all 4 channels, where x is the generated image and t is the target image. It squares the errors, giving more weight to larger errors, and is used as a baseline in this

project.

$$\mathcal{L}_{\text{MSE}}(x) = \frac{1}{HWC} \sum_{h,w,c} \left(x_{hwc} - t_{hwc}\right)^2$$

Huber: Huber loss [9] is a robust loss which combines MSE and MAE. It uses MSE for small errors and switches to MAE for larger errors when δ is crossed. We use a small $\delta = 0.1$ threshold to use MSE for pixel errors smaller than 10% of the full range and MAE for larger errors.

$$\ell_\delta(r) = \begin{cases} \frac{1}{2}r^2 & |r| \le \delta \\ \delta\left(|r| - \frac{1}{2}\delta\right) & |r| > \delta \end{cases}$$

Weighted RGBA: This is a variant of the MSE loss that gives a higher weight to the specified channels: $\mathcal{L} = \frac{1}{HWC} \sum_{h,w,c} w_c (x_{hwc} - t_{hwc})^2$. Weights were manually set to $w = 2$ for RGB and $w = 1$ for A.

Foreground-masked MSE: This loss is a spatially weighted variant of MSE that scales each pixel's contribution by its position in the image. It works similarly to weighted RGBA, however, all channels are weighted equally, pixels inside the body contribute fully, but other pixels contribute less, with background pixels contributing only 20%.

$$\mathcal{L}_{\text{fg}}(x) = \frac{1}{HWC} \sum_{h,w,c} \left(0.2 + 0.8\, t_{hw,\alpha}\right) \cdot \left(x_{hwc} - t_{hwc}\right)^2$$

SSIM: SSIM [7] is a so-called perceptual metric that measures similarity between images by taking into account the luminance, contrast, and structure.

$$\mathcal{L}_{\text{SSIM}}(x) = 1 - \frac{1}{|W|} \sum_{w \in W} \frac{(2\mu_x\mu_t + c_1)(2\sigma_{xt} + c_2)}{(\mu_x^2 + \mu_t^2 + c_1)(\sigma_x^2 + \sigma_t^2 + c_2)}$$

Results

Results. Ensure to perform a correct experimental analysis (e.g. repeated experiments, statistical analysis of results). Verbally describe your results in this section and make sure to reference the corresponding figures/tables.

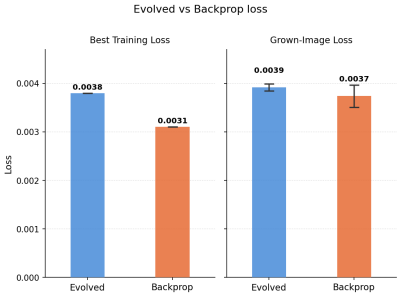

Figure 5: The best loss achieved during training and the averaged loss of the final model after training both of our implementation and the gradient-based implementation after 30 minutes of training time.

While the gradient-based model reaches better results in the same time, the loss of the NCA-ES trained adversarial attack is not too far removed. It is interesting to note that the gradient-based attack has a higher standard deviation than the evolved one.

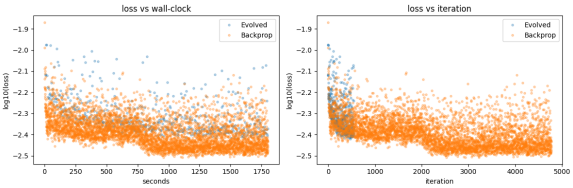

Figure 6: Different ways of measuring the x-axis: time or training steps/generations. On the y-axis, the loss of the gradient-approach is always a bit better. Comparing by training steps/generation, the losses look comparable, though much more computational steps are completed per generation than per training step, making the time-based measurement approach the 'fairest' one.

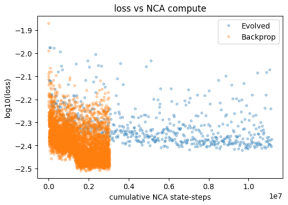

Figure 7: Another way to measure the x-axis: How often does the NCA run? Here the gradient-based approach looks much better.

Figure 8: Left: The target image for our attack. Middle: Lizard grown from pixel with CMA-ES attack (ours). Right: Lizard grown from pixel with gradient-based attack.

Figure 9: Left: fully grown lizard, original target image. Middle: CMA-ES attack on fully grown lizard. Right: gradient-based attack on fully grown lizard

Figure 10: Gradient-based attacked Lizard during growing steps 0, 20, 40, 60, 80, 100, 120

Figure 11: Evolutionary attacked Lizard during growing steps 0, 20, 40, 60, 80, 100, 120

Visually, both the gradient-based attack and our lizard trained with CMA-ES produce noisier results than the trained NCA and the target image, which becomes obvious when looking at them closely, but not when looking at them from afar. In detail, our lizard has sharper, ragged edges and colours like the original green but also a darker blue shining through more. The gradient-attacked lizard has greater colour accuracy and superiour details (such as the position of the eyes & the white line on the back), but seems a bit blurrier. That said, it is not as easy as to say that the gradient-based attack produces a blurred version of the target image, as a blurred version of the target image is of brighter colour, with cleaner fine details (such as the eyes and tail).

Figure 12: For reference: the target image transformed down and then up 4x with linear, none, and cubic interpolation algorithms in GIMP.

Different starter matrices

Instead of starting from identity as attack matrix, we also tried starting from white noise & the 8000 step gradient-optimized matrix. We used the same hyperparameter values we found using optuna to train a 500-generation attack starting from the identify matrix.

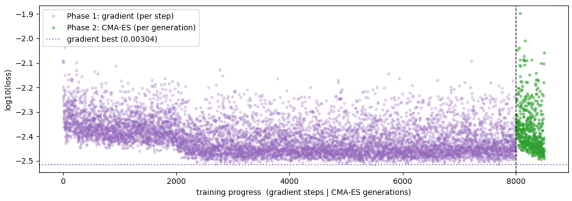

Figure 13: CMA-ES does not visibly improve upon the gradient-baseline training loss within 500 generations. There is a significant jump upwards in loss after the switch, likely caused by sigma0 being too high and too big changes being made.

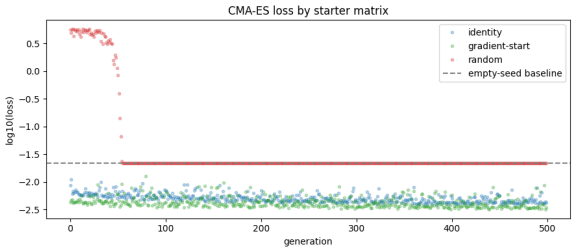

Figure 14: While the model trained starting from the identity matrix and the model trained from the gradient-result-matrix continued improving, the white-noise matrix start only managed to not be worse than producing a white canvas and got stuck there. The baseline here is the loss for the starter image of white canvas with a single black pixel in the middle.

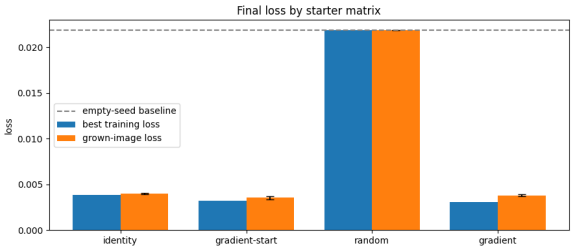

Figure 15: In averaged loss after growing a full lizard from a pixel, the random matrix start sits at the empty baseline. Identity comes third place at 0.0040, gradient-only is second at 0.0038 (+- 0.000086) and the best result comes from further training the gradient output with CMA-ES at 0.0036 (+- 0.000177).

Figure 16: Target (nr.1) and gradient-trained (nr. 4) lizards among CMA-ES trained ones. The third one started training on the gradient-trained matrix. The output from the white-noise trained matrix is just white, while the other ones clearly show a lizard. The Lizard trained from the gradient-trained baseline matrix seems to have the best colour accuracy, while the gradient-trained one has the most detailed white spine.

Results - Losses

Table ?? displays the evaluation metrics for the explored losses.

Loss	SSIM (higher)	LPIPS (lower)	FFT (lower)
MSE	0.9245	0.0959	0.5410
Huber	0.9349	0.0972	0.4949
Weighted RGBA	0.9334	0.1034	0.5439
Foreground-masked MSE	0.9268	0.1173	0.5760
SSIM ⋆	0.9497	0.1970	0.3617

Table 1: Losses with their evaluation metrics. (lower) or (higher) determine if a smaller or larger scores are better. ⋆ means that the lizard did not turn blue.

As can be seen from the table, all losses perform similarly well. However, out of all the losses, Huber performs the best overall, achieving the highest SSIM of 0.9349, the lowest FFT of 0.4949, and the second lowest LPIPS of 0.0972.

While quantitative metrics are a good indicator of performance, they do not fully reflect the actual results. In order to do that, we need to also inspect the produced final images displayed in Figure 17, which shows the mean grown image averaged over 8 rollouts using the final perturbation matrix for each of the losses, along with the target image. All losses, except the SSIM, can successfully recolor the lizard, however, there are visible differences. Huber produces the sharpest looking result consistent with its performance across all metrics, but the limbs and the head have green residue and the edges of the body are much darker compared to others. MSE produces a similar result with slightly better colors. Although it is less sharper and lacks behind in some metrics, the results look to be on par with Huber. The weighted RGBA and the Foreground-masked MSE both produce noticeably blurry results. Comparing the two, weighted RGBA better handles color and does not produce pink eyes. Lastly, although the SSIM has technically the highest SSIM and lowest FFT, it fails the task of turning the lizard blue. This is most likely due to fact that the green lizard to a large degree already satisfies its measures of luminance, contrast, and structure. This is an example of a phenomenon called Goodhart's law [10] which states that when a measure becomes a target, it ceases to be a good measure.

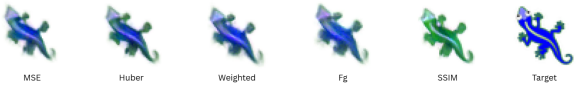

Figure 17: Mean grown image averaged over 8 rollouts using the final perturbation matrix for each of the 5 losses, along with the target image.

Overall, Huber provides the best results closest to the target, both quantitatively and visually. The results show that the selection of loss function affects the quality of the output and the attack in general.

Discussion

Discussion. A thorough discussion based on the analysis of the results, including limitations and future directions (What additional enhancements could be made? What future work should be done?)

Conclusion

Conclusions. Is your line of work worth pursuing?

References

[1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, "Growing neural cellular automata," *Distill*, vol. 5, no. 2, e23, 2020. [Online]. Available: https://distill.pub/2020/growing-ca doi: 10.23915/distill.00023.

[2] Y. Xu, T. Zhang, and S. Süsstrunk, "AdaNCA: Neural cellular automata as adaptors for more robust vision transformer," *Advances in Neural Information Processing Systems*, vol. 37, pp. 25709–25746, 2024.

[3] E. Randazzo, A. Mordvintsev, E. Niklasson, and M. Levin, "Adversarial reprogramming of neural cellular automata," *Distill*, vol. 6, no. 5, e00027-004, 2021. [Online]. Available: https://distill.pub/selforg/2021/adversarial doi: 10.23915/distill.00027.004.

[4] L. Cavuoti, F. Sacco, E. Randazzo, and M. Levin, "Adversarial takeover of neural cellular automata," in *Artificial Life Conference Proceedings 34*, 2022, p. 38. MIT Press.

[5] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.

[6] S. Greydanus, "Studying growth with neural cellular automata," blog post and PyTorch implementation, 2022. [Online]. Available: https://greydanus.github.io/2022/05/24/studying-growth

[7] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[8] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2018, pp. 586–595.

[9] P. J. Huber, "Robust Estimation of a Location Parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964. [Online]. Available: https://doi.org/10.1214/aoms/1177703732

[10] M. Strathern, "'Improving ratings': audit in the British University system," *European Review*, vol. 5, no. 3, pp. 305–321, 1997. [Online]. Available: https://doi.org/10.1002/(SICI)1234-981X(199707)5:3<305::AID-EURO184>3.0.CO;2-4

Appendix

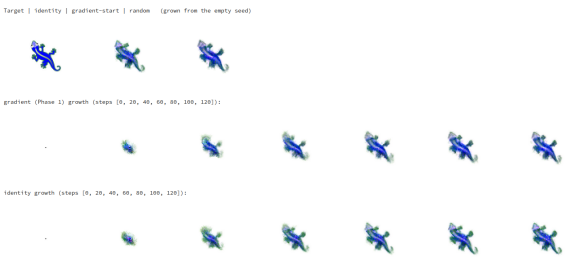

Figure 18: Results from Experiments starting with different matrixes.

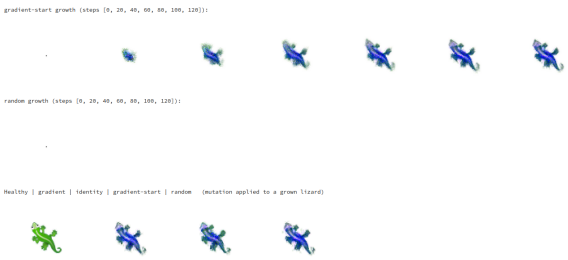

Figure 19

Author Contribution

Author contribution: describe each team member contributions to the project.

Code

Include a link to a github repository with implementation of algorithms developed and used during the project. The repository should include source code, all data necessary to run the program (think instructions for installing external software, package versions/virtual environment, etc); a short manual describing how to use/run the program; a sample run, screenshot, or other indication of system behavior.